

What Does a VPN Actually Hide?

A hands-on AirVPN verification using WireGuard, packet captures, DNS tests, custom nftables kill-switch failure testing, IPv6 enforcement tests, and provider-trust analysis

Kyle Pakhet / Fragmented Sentinel

Independent technical test - 26-28 September 2026 | Version 1.1 | Revised 28 September 2026

Version 1.1 DOI: <https://doi.org/10.5281/zenodo.23027447>

Concept DOI (all versions): <https://doi.org/10.5281/zenodo.22986652>

Source inspiration: NetworkChuck, "VPN myths / what a VPN actually does." [1]

The question

A VPN clearly encrypts traffic between a device and a VPN server. But what does that actually hide from an ISP, what can still leak, and what trust simply moves to the VPN provider? This paper tests those questions rather than accepting either marketing claims or blanket skepticism.

Version 1.1 revision note

This revision corrects the test operating system, explicitly separates the tester-maintained nftables kill switch from AirVPN Eddie/Network Lock, adds controlled IPv6 enforcement testing, adds startup-failure and shutdown A/B tests, documents the wg-quick/systemd-resolved DNS mechanism, publishes a sanitized reproducibility package, and tightens the distinction between provider claims and independently observed evidence.

Abstract

This paper documents a practical verification of an AirVPN connection from a Linux Mint 22.3 x86_64 virtual machine under VMware using an AirVPN-generated WireGuard configuration and `wg-quick`. The investigation began with a simple question: does a personal VPN materially reduce destination information visible to a local observer or ISP, or does it merely move that visibility to the VPN provider? Direct routing inspection, packet capture, DNS analysis, controlled bypass attempts, failure injection, and reboot testing show that both propositions are true.

Normal connected-state testing showed IPv4, IPv6, DNS, hard-coded public DNS, and ordinary HTTPS traffic traversing WireGuard rather than appearing as separate Internet flows on the physical interface. DNS configuration was applied locally by `wg-quick` through the system `resolvconf` compatibility command, which on this host resolved to `/etc/resolv.conf`; WireGuard itself did not negotiate DNS. Plain port-53 DNS was visible on the logical VPN interface but not on the physical interface, where the corresponding activity appeared as encrypted WireGuard UDP.

The most consequential findings involved the tester-maintained system firewall, not AirVPN Eddie or Network Lock. The original persistent nftables ruleset failed at boot because it referenced the VPN interface in a way that required the interface to exist and also contained an obsolete endpoint. Later failure-injection testing showed that systemd ordering alone was not sufficient for fail-closed startup:

INDEPENDENT VPN VERIFICATION

when nftables was deliberately forced to fail, NetworkManager still started and direct Internet traffic escaped. Explicit service dependencies corrected that behavior. Separately, a broad OUTPUT `tcp` state established, related `accept` rule was shown to permit an established VPN TCP flow to emit a FIN over the physical interface during reboot; removing that rule prevented the reproduced shutdown escape.

A controlled IPv6 test also corrected an important limitation in Version 1.0. The original VMware NAT environment had no usable non-VPN IPv6 route, so an earlier `Network unreachable` result could not prove firewall enforcement. Version 1.1 establishes a working temporary IPv6 underlay first, then demonstrates that the hardened nftables policy blocks the same path with zero matching packets observed externally. Server-side trust remains a separate question: AirVPN states that it does not log or inspect user activity, but those claims are not independently established by client-side testing, and RTINGS reports no published independent privacy/no-logs or infrastructure audit. [2][3][4][5]

1. Why test this at all?

The investigation started after watching NetworkChuck test the idea that HTTPS already makes a personal VPN unnecessary. His demonstration showed that HTTPS protects content while still leaving useful destination-level network metadata visible to a local observer; once traffic was encapsulated in WireGuard, the local capture mainly saw the VPN tunnel. The video also emphasized the more important point that a VPN shifts trust from the ISP to the VPN provider. [1]

Rather than debating those claims in the abstract, I used an existing AirVPN WireGuard VM and asked a series of progressively harder questions:

- Does normal IPv4 traffic actually use the VPN?
- Does IPv6 use the VPN, and can a viable non-VPN IPv6 path be blocked?
- Where do DNS queries go, and what is visible inside versus outside the tunnel?
- What actually configures DNS on this WireGuard client?
- What happens if the VPN drops or the firewall fails to start?
- Can an application bypass the tunnel by binding directly to the physical interface?
- Can an application use a hard-coded DNS resolver without exposing the home connection?
- Can an established connection escape during shutdown?
- Does protection survive a normal reboot after hardening?
- What infrastructure sits at the other end of the tunnel, and what does that imply for trust?

2. Test setup and evidence standard

The test system was a Linux Mint 22.3 (Zena) x86_64 virtual machine under VMware, running Linux kernel `7.0.0-34-generic`. Its ordinary interface, `ens33`, used VMware NAT on `192.168.107.0/24`. A separate `airvpn` interface was created by `wg-quick`. The tested AirVPN peer used UDP endpoint `198.44.159.13:1637`, with `AllowedIPs = 0.0.0.0/0, ::/0`, so the intended design was a full-tunnel IPv4/IPv6 VPN. The VPN interface used IPv4 `10.188.221.137/32` and an AirVPN-provided ULA IPv6 address.

The kill switch in this paper is a custom, tester-maintained system-level nftables policy used alongside the AirVPN-generated WireGuard profile. AirVPN Eddie and AirVPN Network Lock were not used or evaluated. Any finding about firewall behavior therefore applies to this Linux Mint + `wg-quick` + nftables configuration unless explicitly stated otherwise.

The evidence standard was deliberately conservative. Results were classified as one of three types:

INDEPENDENT VPN VERIFICATION

1. Directly observed: routing output, nftables state, systemd dependencies, packet captures, DNS responses, controlled interface tests, failure-injection tests, reboot journals, and public-IP observations.
2. Provider-asserted: statements made by AirVPN about logging, server type, privacy practices, or infrastructure.
3. Externally reported: findings or provider responses published by sources such as RTINGS, TorrentFreak, Akamai, and ARIN.

Why this matters

A client-side packet capture can prove that traffic did or did not leave a specific interface during the observation window. It cannot prove that a VPN provider never logs traffic after it reaches provider infrastructure. That evidence boundary is central to the conclusions in this paper.

3. The custom nftables kill switch was initially broken

The first major result concerned the custom persistent nftables firewall. The original configuration referenced ``airvpn`` with interface-object matching that required the interface to exist when nftables loaded. During boot, ``wg-quick`` had not yet created the interface, so the persistent ruleset failed to load. The ruleset also contained an obsolete AirVPN endpoint rather than the endpoint currently configured in WireGuard.

With the custom firewall absent, deliberately stopping WireGuard allowed ordinary routing to fall back to the VMware NAT gateway and restored direct Internet access. This demonstrated failure of the local custom kill-switch configuration; it was not a test of AirVPN Network Lock.

```
VPN stopped
-> WireGuard policy route removed
-> VMware NAT route became usable
-> ordinary non-VPN Internet access continued
-> custom kill-switch objective was not met
```

The persistent rules were repaired by using interface-name matching (``iifname`` / ``oifname``) so the firewall could load before the VPN interface existed, updating the permitted WireGuard endpoints, retaining a default-drop OUTPUT policy, and allowing ordinary Internet traffic only through ``airvpn``. Later testing identified one additional hardening requirement: the OUTPUT chain must not contain a broad ``ct state established,related accept`` rule, because that rule enabled a shutdown escape described in Section 7.

4. DNS configuration and packet path

Version 1.0 correctly observed that DNS traffic remained inside the VPN, but its description of how the resolver was configured was too loose. WireGuard itself does not negotiate or "push" DNS settings. The tested local ``wg-quick`` profile contained ``DNS = 10.128.0.1, fd7d:76ee:e68f:a993::1``. The ``wg-quick(8)`` interface format treats ``DNS`` as a helper option and invokes ``resolvconf`` when the interface is brought up or down. [11]

On this Linux Mint system, ``/usr/sbin/resolvconf`` was a symbolic link to ``/usr/bin/resolvectl``, both supplied by ``systemd-resolved``. The boot journal recorded ``wg-quick`` invoking ``resolvconf -a airvpn -m 0 -x``. ``systemd-resolved`` then associated the two AirVPN resolver addresses with the ``airvpn`` link and installed the route-only domain ``~.``, which is used for DNS routing decisions. [12] Stopping ``wg-`

INDEPENDENT VPN VERIFICATION

quick@airvpn` removed the AirVPN DNS state; restarting it restored the same resolver addresses and `~.` domain.

The packet-path experiment captured DNS simultaneously on the logical `airvpn` interface and the physical `ens33` interface. A unique NXDOMAIN query and a normal `example.com` lookup produced ordinary A/AAAA port-53 traffic between `10.188.221.137` and `10.128.0.1` inside the VPN. On `ens33`, no port-53 DNS was observed; corresponding activity appeared only as WireGuard UDP between the VM and `198.44.159.13:1637`.

```
Inside airvpn:
10.188.221.137:* -> 10.128.0.1:53   DNS A/AAAA queries
10.128.0.1:53    -> 10.188.221.137:* DNS responses

Outside ens33:
no port-53 packets observed
192.168.107.128:* <-> 198.44.159.13:1637   WireGuard UDP
```

This distinction matters when interpreting resolver tools. `systemd-resolved` reported that the lookup was not acquired via an encrypted DNS transport, because the resolver transaction itself used ordinary DNS rather than DNS-over-TLS or a similar DNS-specific encryption method. That does not mean the query crossed the physical network in plaintext: the simultaneous packet captures show that the ordinary DNS packet existed inside the WireGuard tunnel and was encrypted before traversing `ens33`.

5. What the ISP-side capture could still see

A broader packet capture was taken on the external interface while HTTPS requests were made to multiple websites. Instead of separate connections to those destinations, the outside interface showed UDP traffic between the VM and the AirVPN WireGuard endpoint. The destination DNS names and destination-IP connections were not exposed as separate flows on that interface.

Observable outside the tunnel	Not directly observable outside the tunnel
AirVPN endpoint IP	DNS queries carried inside WireGuard
UDP protocol / tunnel port	Destination IP flows carried inside WireGuard
Packet timing and direction	TLS SNI belonging to inner HTTPS sessions
Packet sizes and traffic volume	HTTPS content, URLs, passwords, messages
Session duration / traffic bursts	Application-level traffic as separate Internet flows

This is the first major qualification to the phrase "a VPN hides your metadata." It hides a large amount of destination-related metadata from the local/ISP side, but it does not make the connection invisible. Timing, packet size, direction, duration, and the VPN endpoint remain observable. A sufficiently capable adversary with visibility on both sides of a VPN may attempt traffic correlation without decrypting WireGuard.

6. Deliberate bypass tests and a valid IPv6 baseline

Normal routing tests do not answer what happens when software deliberately tries to avoid the expected path. The VM was therefore tested as if an application were misconfigured or actively attempting to bypass the VPN. Version 1.1 also distinguishes two different IPv6 questions: whether IPv6 uses AirVPN when the VPN is active, and whether the firewall blocks a viable non-VPN IPv6 path on the physical interface.

Test	Method	Result
Force HTTPS through `ens33`	`curl --interface ens33 ...`	Timed out; no Internet packets captured on `ens33`.

INDEPENDENT VPN VERIFICATION

Test	Method	Result
Force IPv4 ICMP through `ens33`	`ping -I ens33 1.1.1.1`	100% packet loss under hardened firewall.
Force local VMware DNS	`dig @192.168.107.2 example.com`	Timed out; no port-53 packets left `ens33`.
Hard-coded public DNS	`dig @1.1.1.1 example.com`	Succeeded, but capture showed the query traversed `airvpn`.
IPv6 through AirVPN	`curl -6 --interface airvpn ...`	Succeeded with an AirVPN/tzulo IPv6 exit.
Original direct IPv6 attempt	`ping -6 -I ens33 ...`	Failed, but underlay had no usable non-VPN IPv6 route; not proof of firewall enforcement.
Controlled IPv6 allow baseline	Temporary `fd42:107::/64` underlay + temporary nftables allow	4/4 ICMPv6 replies; host capture observed requests/replies.
Controlled IPv6 blocked state	Same address, route, interface, destination; temporary allow removed	0/4 replies and zero matching packets on host `vmnet8` capture.

For the controlled IPv6 test, the VMware host temporarily used `fd42:107::1/64` on `vmnet8` and the VM used `fd42:107::128/64` on `ens33`. With a temporary IPv6 exception in nftables, the path succeeded and was externally observed. After removing only that exception, the identical ping failed and the externally positioned host capture saw zero matching packets. This establishes that the tested nftables policy blocked a known-working non-VPN IPv6 path. It does not constitute testing against a native public IPv6 Internet uplink.

The hard-coded `1.1.1.1` result is also useful. It demonstrates the difference between a DNS-policy bypass and a VPN leak. An application can ignore AirVPN's DNS resolver and query another public resolver, but in this configuration the packet still travelled through WireGuard; the public resolver therefore saw the VPN exit rather than the home connection.

7. Startup and shutdown fail-closed verification

Version 1.0 inferred fail-closed startup primarily from successful boot ordering: nftables completed before normal networking and `wg-quick`. Subsequent failure injection showed that this inference was too strong. Ordering a service before networking is not the same as requiring networking to fail when the firewall fails.

To test the failure case, nftables startup was deliberately replaced with a failing command for one controlled boot while an external host captured the VMware network and a canary repeatedly attempted direct IPv4 traffic over `ens33`. Under the original dependencies, nftables failed but NetworkManager still started, obtained `192.168.107.128`, and direct public traffic was externally observed, including the canary ICMP, DNS to the VMware resolver, and operating-system connectivity traffic.

The permanent fix added an explicit `Requires=nftables.service` / `After=nftables.service` dependency to NetworkManager. A second dependency made `wg-quick@airvpn` require both nftables and NetworkManager. In the final deliberate firewall-failure boot, nftables was failed, NetworkManager remained inactive, `wg-quick@airvpn` remained inactive, `ens33` had no configured IPv4 address, `airvpn` did not exist, all direct-interface canary probes failed, and no public Internet traffic was observed during the failed state. A subsequent normal reboot confirmed that nftables, NetworkManager, WireGuard policy routing, DNS, and AirVPN connectivity all returned normally.

Shutdown testing exposed a different condition. During a full reboot, an established TCP connection originally routed through WireGuard emitted a FIN packet directly over the physical VMware interface after the tunnel routing state was removed while the custom OUTPUT chain contained a broad `ct state established,related accept` rule. Repeating the same known-flow test after removing that OUTPUT rule prevented the flow from appearing outside the tunnel. The persistent hardened ruleset was updated

INDEPENDENT VPN VERIFICATION

accordingly and a final reboot capture did not show the known test flow or WireGuard-source traffic escaping on the physical interface.

Scenario	Observed result	Interpretation
Firewall forced to fail; original dependencies	NetworkManager started and direct Internet traffic was externally captured.	Systemd ordering alone was not fail-closed.
Firewall forced to fail; hardened dependencies	NetworkManager and `wg-quick` remained inactive; canary failed; no public traffic during failed state.	Explicit service requirements enforced fail-closed startup.
Full reboot with broad OUTPUT established-state rule	Known VPN TCP flow emitted a direct FIN on the physical network.	Broad OUTPUT conntrack exception enabled shutdown escape.
Full reboot after removing broad OUTPUT established-state rule	Known flow not observed outside tunnel.	Reproduced escape condition was removed.
Final normal reboot	Firewall, networking, WireGuard, routing and AirVPN connectivity restored normally.	Hardening did not prevent normal operation.

Scope of the firewall finding

These startup and shutdown results apply to the tested Linux Mint system using `wg-quick` plus a separately maintained nftables policy. They are not measurements of AirVPN Eddie or AirVPN Network Lock. RTINGS tests a different client/OS arrangement and reports an AirVPN reboot leak under its Windows-based kill-switch methodology. [5][13]

8. Identifying the AirVPN infrastructure

The tested WireGuard endpoint resolved into address space associated with tzulo / AS11878, and the public VPN exit used a different address in the same broader provider network. ARIN registration data and routing information support attribution of the surrounding network, but they do not establish physical co-location, server ownership, or whether entry and exit services run on the same physical host. [8] The directly supported finding is narrower: the tested path used distinct entry and exit IPv4 addresses.

The DNS path yielded another useful result. Akamai's `whoami.ds.akahelp.net` diagnostic reports the public source address of the recursive resolver contacting Akamai and can report EDNS Client Subnet (ECS) when present. [6] When queried through AirVPN's internal resolver, the observed resolver egress was in related AirVPN/tzulo IPv6 space and no ECS field was observed in that query.

CHAOS-class DNS queries returned `version.bind = "AirVPN DNS"` and `id.server = "unurgunite.airservers.org"`. The resolver therefore identified itself with an AirVPN node name while suppressing a normal software-version string. The resolver egress and web IPv6 exit were different addresses within the same `/64`; that is consistent with closely related network infrastructure but does not prove that the services occupy the same physical machine or facility.

9. Where the trust moves

At this point the client-side question was largely answered: the local/ISP side saw the WireGuard relationship rather than the separate DNS and destination flows inside it. But encryption between the client and the VPN server does not make the provider blind.

Once traffic reaches the VPN server, the WireGuard layer must be processed so that packets can be forwarded. For ordinary HTTPS, end-to-end TLS still protects page contents, passwords, and messages from the VPN provider. The VPN provider is nevertheless technically positioned to observe or correlate

information such as the incoming VPN connection, destination IP addresses, timing, traffic volume, and DNS queries when its resolver is used.

The trust shift

Before the VPN, the ISP has direct visibility into much of the destination-level network metadata. With the VPN, the ISP mainly sees the encrypted tunnel to AirVPN. AirVPN then occupies the privileged network position at the other end. The visibility is not destroyed; much of it is relocated.

10. Provider claims, third-party reporting, and independent evidence

AirVPN publicly states that it does not monitor or log users' online activities. AirVPN staff have separately stated that the service does not retain logs capable of identifying a customer or VPN client IP address and does not monitor or inspect client traffic. [2][3] These statements describe AirVPN's published policy and representations; they were not independently verified by the client-side experiments in this paper.

TorrentFreak's 2026 VPN privacy survey records AirVPN stating that it does not retain data that would allow an IP address and timestamp to be matched to a current or former user, that it uses no monitoring tools for connection enforcement, and that it operates real bare-metal servers rather than VPS or virtual locations. [4] These are answers supplied by AirVPN and published by TorrentFreak, not the results of an independent infrastructure or no-logging audit.

RTINGS provides a different category of evidence. Its current AirVPN review reports no ordinary IP or DNS leaks while connected, reports leakage during its system-reboot kill-switch test, and states that AirVPN has not published independent third-party audits verifying its privacy/no-logs policy or security infrastructure. [5] RTINGS' reboot methodology uses a Windows test system while a separate Linux system captures traffic externally, so its result should not be treated as a direct reproduction of this Linux Mint + `wg-quick` + custom nftables experiment. [13]

The absence of a published independent audit does not establish that AirVPN logs traffic. It means the provider's server-side retention claims remain outside the independent evidence available to this study. Client-side packet capture can establish what crosses the local physical interface; it cannot observe provider-side logging, database retention, administrative access, or upstream network collection.

There is also behavioural evidence about AirVPN's privacy posture. In 2024, AirVPN stopped accepting Italian residents after Italy's Piracy Shield framework extended rapid blocking obligations to VPN/DNS intermediaries. TorrentFreak reported that AirVPN chose withdrawal rather than implementing those blocking requirements. [7] That behaviour may inform a personal trust assessment, but it is not proof of no logging.

11. Evidence matrix

Claim	Evidence type	Confidence / scope
IPv4 uses the VPN in the tested configuration	Routing + external packet capture	Directly demonstrated for this VM/path.
IPv6 uses AirVPN when the VPN is active	Routing + public IPv6 observation	Directly demonstrated for this VM/path.
Custom firewall blocks a viable non-VPN IPv6 path	Controlled ULA A/B test + external capture	Directly demonstrated on VMware underlay; not native public IPv6.
DNS configuration comes from local `wg-quick` integration	Sanitized profile + boot journal + resolver state	Directly demonstrated on this client.
DNS is contained inside WireGuard	Simultaneous inside/outside packet capture	Directly demonstrated for tested queries.

INDEPENDENT VPN VERIFICATION

Claim	Evidence type	Confidence / scope
Physical-interface IPv4 bypass is blocked	Direct adversarial test + external capture	Directly demonstrated under hardened rules.
Startup fails closed after dependency hardening	Deliberate nftables failure + canary + external capture	Directly demonstrated in controlled failure test.
Broad OUTPUT established-state rule enabled shutdown escape	Known-flow full-reboot A/B packet capture	Directly demonstrated for tested configuration.
Hardened configuration survives normal reboot	Normal reboot + service/routing/capture verification	Directly demonstrated for tested boot.
Separate entry/exit IPv4 addressing on tested path	Endpoint/public-exit comparison	Directly demonstrated for this path.
AirVPN resolver did not send ECS in tested diagnostic	Akamai diagnostic	Strong for that query only.
AirVPN states it does not retain identifying traffic logs	Provider policy/statements; TorrentFreak records provider response	Not independently verified by this study.
AirVPN states it operates bare-metal servers	Provider statement recorded by TorrentFreak	Not independently verified by this study.
All AirVPN nodes behave like this tested path	Not tested	Unknown.

12. What this paper does not prove

This was a point-in-time test of one Linux Mint 22.3 VM, one AirVPN-generated WireGuard configuration, a small number of AirVPN paths/endpoints, and a custom local firewall. It was not an audit of AirVPN's production environment. The following remain outside the evidence collected here:

- Whether AirVPN's deployed production systems implement its published retention and no-logging policies exactly as described. The experiments had no administrative, storage, or server-side visibility into AirVPN infrastructure.
- Whether every AirVPN server uses identical logging, DNS, networking, and hardening configuration.
- Whether the tested infrastructure is owned by AirVPN, rented directly from a network provider, colocated, or supplied through another dedicated arrangement.
- Whether related AirVPN entry, exit, and DNS addresses occupy the same physical server or facility.
- What access hosting/network providers may have at switching, transport, server, or upstream network layers.
- Whether a capable adversary observing both sides of the VPN could successfully correlate traffic.
- Whether IPv6 firewall behavior would be identical on a native public IPv6 uplink; the controlled non-VPN IPv6 enforcement test used a VMware ULA underlay.
- Whether AirVPN Eddie or Network Lock behaves like this custom nftables setup; those client features were not used or tested.
- Whether browser cookies, authenticated logins, fingerprinting, application telemetry, or other non-network identifiers can identify the user. A VPN is not an anonymity system by itself.

13. Conclusion

The original question was whether a personal VPN meaningfully hides useful information from an ISP or merely moves the problem somewhere else. The testing supports both propositions.

On the client side, the tested full-tunnel configuration materially reduced what the physical network could observe. IPv4, IPv6, DNS, hard-coded public DNS, and ordinary HTTPS traffic were carried through WireGuard; forced physical-interface attempts were blocked by the hardened firewall; a controlled non-VPN IPv6 path was shown to be blocked; deliberate firewall-startup failure was made fail-closed through explicit systemd dependencies; and a reproduced shutdown escape was removed by eliminating a broad OUTPUT established-state exception.

INDEPENDENT VPN VERIFICATION

Those results also show why "the VPN is connected" is an insufficient security test. Version 1.0 correctly identified a broken persistent firewall, but Version 1.1 found two subtler issues that only appeared under failure injection and full-reboot testing: service ordering alone did not guarantee fail-closed startup, and a seemingly reasonable conntrack exception could let an established connection emit traffic after the tunnel was removed. Verification has to include failure transitions, not only the final steady state.

AirVPN's observable client-side behavior in this experiment was consistent with the tested privacy model: the local observer saw the WireGuard tunnel rather than separate DNS and destination flows, and the tested resolver/routing path behaved as documented. These observations do not verify AirVPN's server-side no-logging policy. AirVPN states that it does not log or inspect client activity, and TorrentFreak records similar provider representations. [2][3][4] RTINGS reports that AirVPN has not published an independent third-party privacy/no-logs or infrastructure audit. [5] This does not demonstrate that logging occurs; it means provider-side retention remains a trust claim outside what this client-side study can independently establish.

Bottom line

A VPN does not eliminate trust. It relocates and concentrates it. Direct testing can reduce blind trust on the client side; clear evidence boundaries and independent provider-side verification can reduce it further. In this test, AirVPN materially reduced ISP-visible destination metadata, while the remaining uncertainty was concentrated in server-side retention practices and the surrounding hosting/network environment.

14. Reproducibility and evidence package

Version 1.1 is accompanied by a sanitized reproducibility package containing the test environment, package versions, routing state, sanitized WireGuard profile, custom nftables policy, effective systemd units/dependencies, resolver state, reproduction instructions, an evidence index, and reviewed packet-capture artifacts or summaries. Private WireGuard key material, the tester's home public IP, local host identifiers, and unnecessary raw capture metadata are intentionally excluded.

```
Public package:
  airvpn-reproducibility-v1.1-public-20260928.tar.gz

SHA256:
  093d9a8c186d9cfe04513ef8c55bd1d7c43888c422a001d8dcfdc15c296421fd
```

The public package directly includes the purpose-built RAW-IP DNS capture from inside WireGuard and the zero-packet blocked IPv6 capture. Two useful Ethernet captures are represented publicly by sanitized human-readable summaries because the originals contain local VMware layer-2 identifiers; SHA256 hashes of those preserved originals are published in the evidence index. Startup/shutdown research captures are similarly preserved privately and identified by hashes where publishing the complete raw capture would disclose unrelated boot traffic or unnecessary local metadata.

The reproduction guide also records an important methodological distinction: absence of packets on an interface is only meaningful when the path is known to be viable. That principle motivated the controlled IPv6 A/B test and the explicit startup-failure injection used in this revision.

15. Practical takeaway: verify your own VPN

The most transferable lesson was not specific to AirVPN. A VPN can appear healthy while a kill switch, dependency, resolver integration, or transition-state control is broken. A small verification routine provides much stronger assurance than a green "connected" icon.

- Confirm IPv4 and IPv6 route selection while the VPN is active.
- Capture DNS on both the logical VPN interface and the physical interface while generating fresh lookups.
- Determine which local component actually configures DNS; do not assume the VPN protocol negotiated it.
- Stop the VPN and confirm that Internet traffic fails closed.
- Bind test traffic directly to the physical interface and verify that it cannot escape.
- For IPv6, establish a known-working non-VPN path before claiming that a firewall blocked it.
- Test hard-coded public DNS and distinguish resolver-policy bypass from tunnel escape.
- Test startup failure, not just successful boot ordering.
- Test full shutdown/reboot with an established flow, not only manual interface shutdown.
- Use an external capture point where possible so that "nothing left the host" is independently observable.
- Preserve software versions, effective service dependencies, sanitized configurations, packet captures, and SHA256 manifests.
- Separate provider marketing/policy statements from directly testable behavior and independent audit evidence.

Appendix A - Representative test commands

```
# Route selection
ip route get 1.1.1.1
ip -6 route get 2606:4700:4700::1111
ip -4 rule show
ip -6 rule show

# DNS state and mechanism
resolvectl status
resolvectl dns
resolvectl domain
sudo journalctl -b -u wg-quick@airvpn.service --no-pager

# DNS outside and inside the tunnel
sudo tcpdump -pni ens33 'port 53'
sudo tcpdump -pni airvpn 'port 53'

# Force physical-interface bypass attempts
curl --interface ens33 --connect-timeout 5 https://ifconfig.me
ping -I ens33 -c 3 1.1.1.1

# Local DNS bypass
DIG='dig @192.168.107.2 example.com +time=2 +tries=1'
$DIG

# Hard-coded public DNS
dig @1.1.1.1 example.com +time=2 +tries=1

# Controlled IPv6 underlay example
sudo ip -6 addr add fd42:107::128/64 dev ens33 nodad
ping -6 -c 4 -I ens33 fd42:107::1

# Firewall and service dependency inspection
sudo nft -a list ruleset
systemctl show nftables.service -p Requires -p Wants -p After -p Before -p DefaultDependencies
systemctl show NetworkManager.service -p Requires -p Wants -p After -p Before -p
DefaultDependencies
systemctl show wg-quick@airvpn.service -p Requires -p Wants -p After -p Before -p
DefaultDependencies

# AirVPN DNS identity / resolver diagnostics
dig +short TXT whoami.ds.akahelp.net @10.128.0.1
dig @10.128.0.1 CH TXT version.bind
dig @10.128.0.1 CH TXT id.server

# Evidence integrity
sha256sum -c SHA256SUMS
```

Failure-injection and shutdown tests can intentionally disrupt networking. They should be performed only on a disposable or recoverable test system with an external capture point and a known recovery path. The reproducibility package documents the tested configuration and evidence without encouraging blind application of the same service overrides to unrelated systems.

References

[1] NetworkChuck - source video that prompted the investigation

<https://www.youtube.com/watch?v=axfNxZ1R6C4>

[2] AirVPN - official service claims and features

<https://airvpn.org/>

INDEPENDENT VPN VERIFICATION

[3] AirVPN staff - "Logging on VPN servers?"

<https://airvpn.org/forums/topic/9717-logging-on-vpn-servers/?comment=11242&do=findComment>

[4] TorrentFreak - "Which VPN Providers Take Privacy Seriously in 2026?"

<https://torrentfreak.com/best-vpn-anonymous-no-logging/>

[5] RTINGS - AirVPN review (Test Bench v0.11; privacy, leak, and kill-switch findings)

<https://www.rtings.com/vpn/reviews/airvpn/airvpn>

[6] Akamai - resolver WhoAmI / ECS diagnostic documentation

<https://www.akamai.com/blog/developers/introducing-new-whoami-tool-dns-resolver-information>

[7] TorrentFreak - AirVPN withdrawal from Italy over Piracy Shield requirements (2024)

<https://torrentfreak.com/airvpn-stops-serving-italians-due-to-piracy-shield-blocking-requirements-240206/>

[8] ARIN - Registration Data Access Protocol (RDAP)

<https://www.arin.net/resources/registry/whois/rdap/>

[9] AirVPN - example server announcement describing no logs and separate entry/exit IP addresses

<https://airvpn.org/forums/topic/74049-new-10-gbits-server-available-se-and-replacement/>

[10] WireGuard - official quick-start documentation

<https://www.wireguard.com/quickstart/>

[11] WireGuard tools - wg-quick(8) manual (DNS/resolvconf behavior)

<https://git.zx2c4.com/wireguard-tools/tree/src/man/wg-quick.8>

[12] systemd-resolved - resolver/routing-domain documentation

<https://www.freedesktop.org/software/systemd/man/systemd-resolved.html>

[13] RTINGS - Kill Switch Robustness methodology

<https://www.rtings.com/vpn/tests/kill-switch-robustness>

Publication note

This document intentionally omits the tester's home public IP address, account information, private cryptographic material, and unnecessary local identifiers. It is published under the creator name Kyle Pakhet / Fragmented Sentinel. Public AirVPN infrastructure addresses are included only where they materially explain the tested network path. Version 1.1 reflects observations made 26-28 September 2026 and should not be read as a permanent certification of AirVPN, of every server in its network, or of AirVPN Eddie/Network Lock.

Version 1.1 supersedes the technical conclusions of Version 1.0 where this revision explicitly narrows or corrects them, especially the operating-system identification, IPv6 enforcement claim, startup-order inference, DNS-configuration mechanism, and scope of the custom kill-switch comparison. Version 1.0 remains available at DOI 10.5281/zenodo.22986653.

Suggested citation: Kyle Pakhet / Fragmented Sentinel. (2026). What Does a VPN Actually Hide?

Independent Verification of AirVPN, WireGuard, DNS and Kill-Switch Behaviour (Version 1.1). Zenodo.

<https://doi.org/10.5281/zenodo.23027447>